RENESAS

Altium

The Solution: Hardware Timers

Independent Hardware Peripherals: Timers are dedicated circuits inside the MCU that

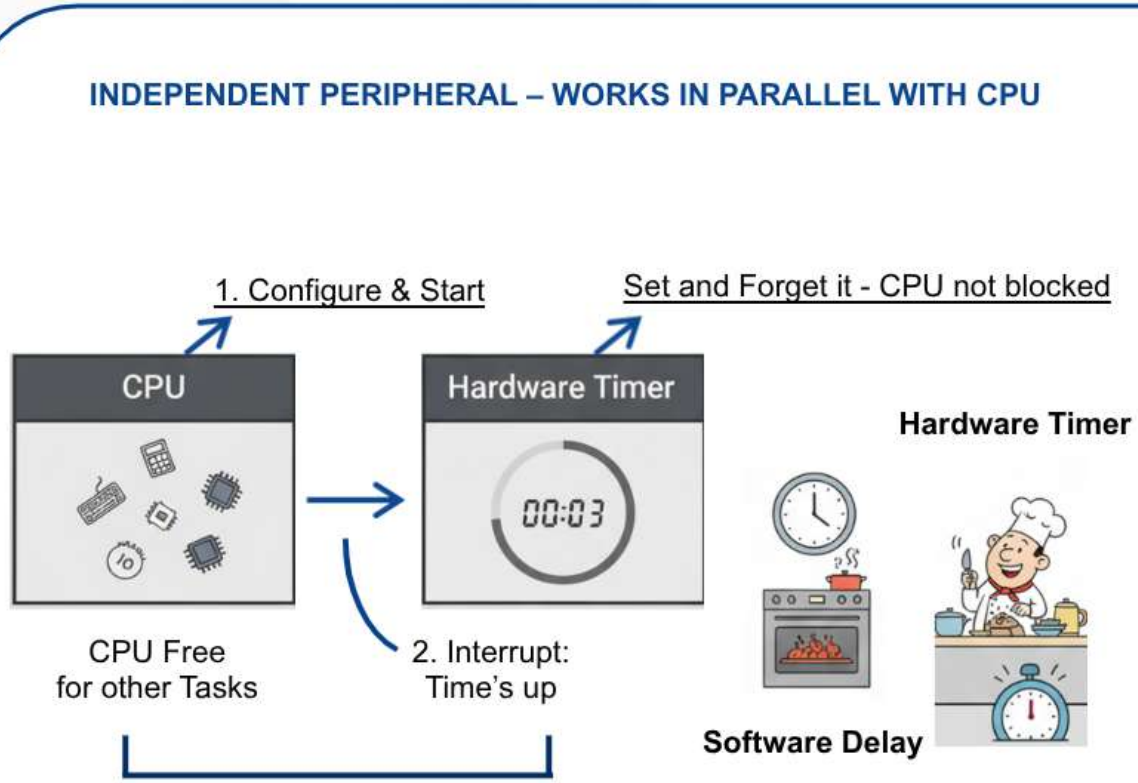
- Operate independently of the CPU core.

The "Set It and Forget It" Principle:

- 1. The CPU configures the timer (sets a time limit).
- 2. The CPU starts the timer.
- 3. The CPU is now free to perform other tasks.
- 4. When the timer finishes, it notifies the CPU with an interrupt.

The Kitchen Analogy:

- **Software Delay:** A chef staring at the clock on the wall while a cake bakes. The chef can do nothing else.
- **Hardware Timer:** A chef sets a kitchen timer and walks away to prepare other dishes. The timer's bell (the interrupt) calls the chef back when the cake is done.

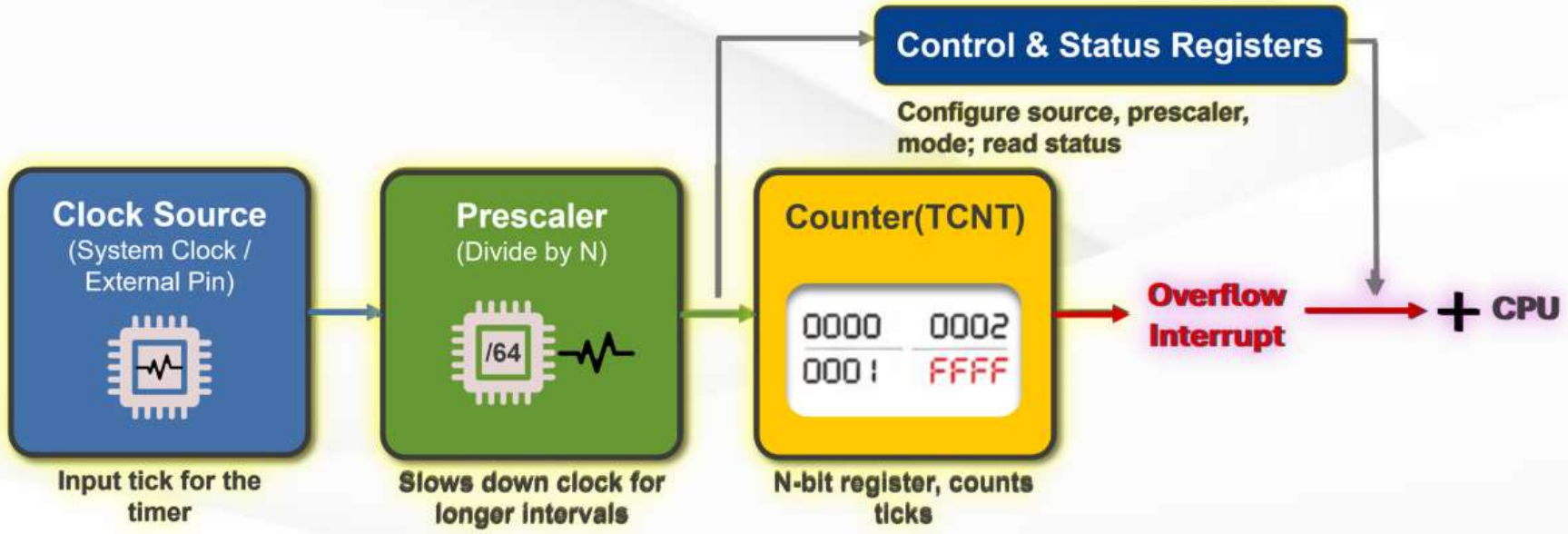


RENESAS

Altium

Anatomy of a Microcontroller Timer

- **Clock Source:** The "tick" for the timer. Can be the main system clock or an external signal.
- **Prescaler:** A configurable frequency divider. It slows down the clock, allowing the timer to measure longer intervals.
- A prescaler of 64 means the counter increments only once for every 64 clock ticks.
- **Counter Register (TCNT):** The core of the timer. An N-bit register (e.g., 16-bit, 32-bit) that counts up or down with each tick from the prescaler. When it reaches its maximum value, it "overflows".
- **Control & Status Registers:** Software-accessible registers to configure the timer's mode, start/stop it, and check its status (e.g., has an overflow occurred?).



RENESAS

Altium

Interacting with the Timer: Polling vs. Interrupts

Aspect	Polling Method	Interrupt Method
Concept	The CPU repeatedly checks a status flag in a loop.	The timer hardware automatically triggers a special function (ISR) when the event occurs.
Example	`while (TIMER_FLAG == 0) { // Do nothing }`	`void TIMER_ISR() { // Event handling code }`
CPU Usage	High (busy-waiting).	Very Low (CPU is free until interrupt).
Responsiveness	Poor. The CPU is blocked.	Excellent. Main code is not blocked.
Analogy	Constantly asking "Are we there yet?" on a road trip.	The driver tells you, "I'll wake you up when we get there."



RENESAS

Altium

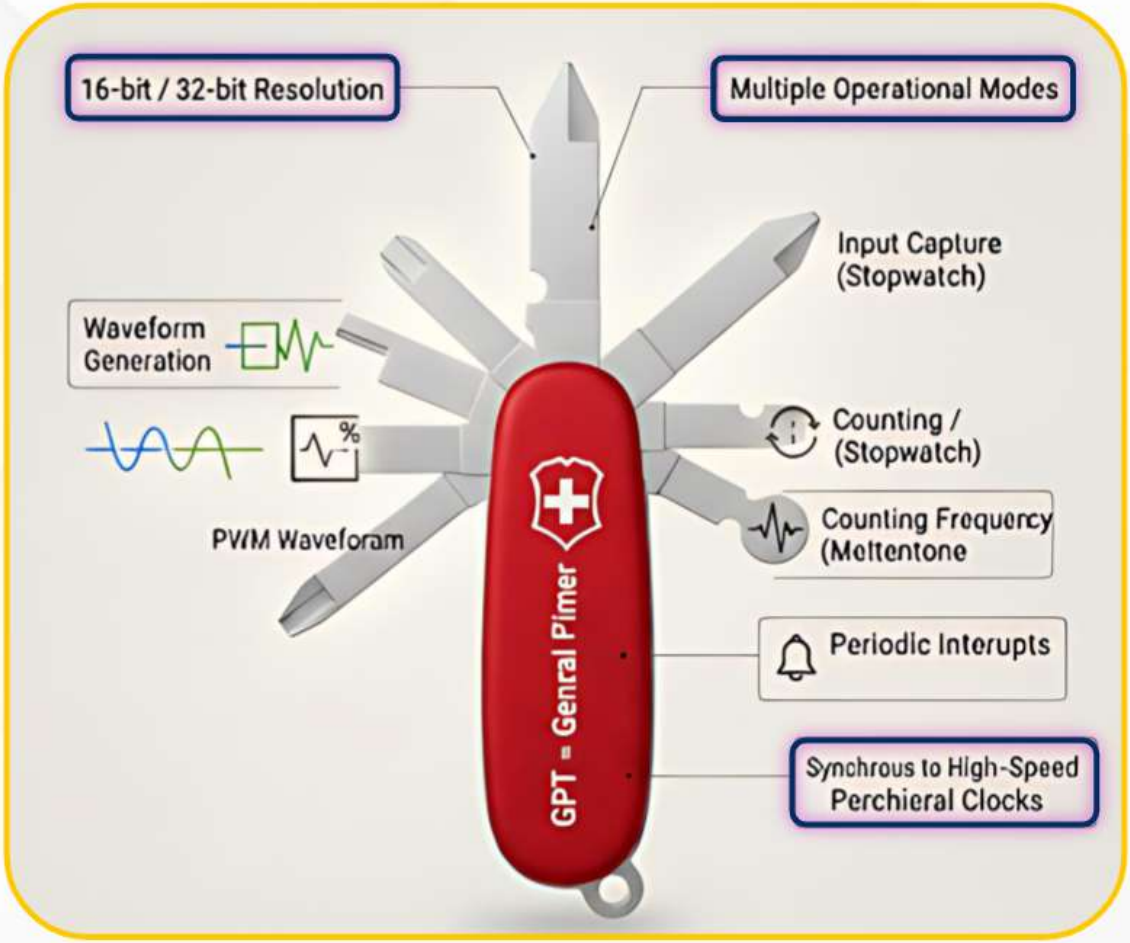
The Workhorse: General Purpose Timer (GPT)

Definition: A highly flexible, powerful, and configurable timer peripheral found in most modern micro-controllers.

Purpose: Designed to handle a wide variety of timing tasks, from simple delays to complex waveform generation

Key Characteristics:

- Typically 16-bit or 32-bit resolution.
- Multiple, feature-rich operational modes.
- Synchronous to the main peripheral clocks (high speed, high power).



RENESAS

Altium

Key GPT Operational Modes

Periodic Mode: Generates a recurring interrupt at a fixed interval. Used for tasks like sampling a sensor every 10ms.

One-Shot Mode: Generates a single interrupt after a specified delay, then stops. Used for timeout mechanisms.

Input Capture Mode: On an external signal edge (e.g., rising edge), the timer's current count value is "captured" into a register. Used to measure pulse widths or signal frequencies.

Output Compare Mode: When the counter value matches a pre-set "compare" value, it can trigger an interrupt or change an output pin's state (set HIGH, set LOW, or TOGGLE).

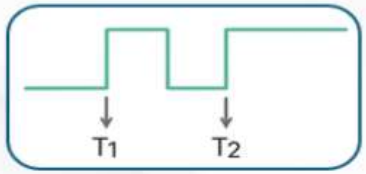
PWM Mode: A specialized form of Output Compare that generates a continuous square wave with a programmable duty cycle. Essential for analog control.



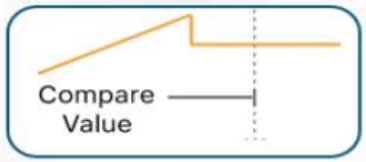
Periodic: Recurring events
Interrupts at fixed intervals



One-Shot: Single interrupt
then stop



Input Capture: Capture counter
value on event edges



Output Compare: Trigger/toggle
when counter = compare

How it Works: GPT Calculations

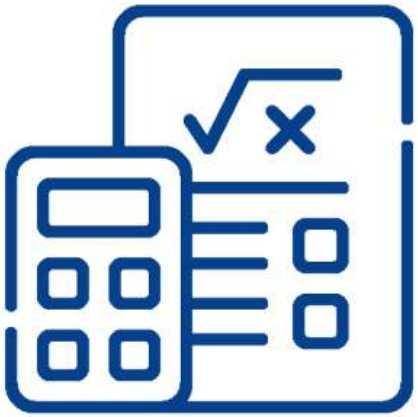
How it Works: GPT Calculations (Periodic Interrupt)

- **The Goal:** To configure the timer to overflow (and generate an interrupt) at a precise, desired time interval.

The Core Formula:

$$T_{timer} = T_{clk} \times \text{Prescaler}(\text{Autoreload_Value}+1)$$

- **Where**:
  - T_{timer}: The desired timer period (e.g., 100 ms).
  - T_{clk}: The period of the timer's source clock ($1/f_{clk}$).
  - **Prescaler**: The clock division factor (e.g., 1, 8, 64...).
  - **Autoreload_Value**: The value the timer counts up to. This is what we need to calculate.



Rearranged to solve for the Auto-reload Value:

$$\text{Autoreload_Value} = \frac{T_{timer}}{T_{clk} \times \text{Prescaler}} - 1$$

Important Note:
The +1 and -1 account for the fact that the timer's count includes the zero state.

0:54 / 0:54

RENESAS

Altium

How it Works: GPT Calculations

Step 1: Find the Source Clock Period (T_{clk}).

- $T_{clk} = 1 / f_{clk} = 1 / 60{,}000{,}000\ \text{Hz} = 16.67\ \text{ns}$

Step 2: Choose a Prescaler. A 32-bit counter can hold a very large value, but using a prescaler improves resolution for some applications and keeps numbers manageable. Let's choose 64.

Step 3: Calculate the Timer's "Tick" Period.

$$T_{tick} = T_{clk} \times \text{Prescaler} = 16.67\ \text{ns} \times 64 = 1.067\ \mu\text{s}$$

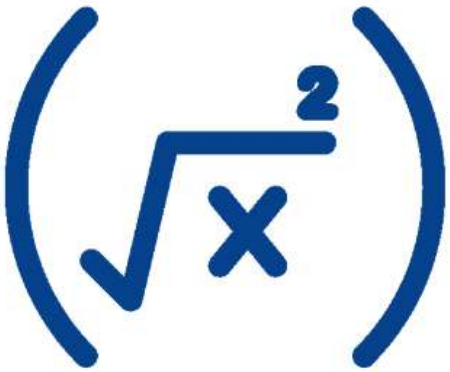
Step 4: Calculate the Required Number of Ticks:

- $RequiredTicks = T_{timer} / T_{tick} = 100\ \text{ms} / 1.067\ \mu\text{s} = 93{,}750$

Step 5: Calculate the Autoreload Value:

$$\text{Autoreload_Value} = \text{Required Ticks} - 1 = 93{,}750 - 1 = 93{,}749$$

- This value is well within the range of a 32-bit register (232−1).



How it Works: GPT Calculations

How it Works: PWM Mode

Concept: Pulse Width Modulation (PWM) controls the average power delivered to a load by varying the ON-time vs. OFF-time of a high-frequency square wave.

Key Terms:

- **Period:** The total time for one ON/OFF cycle. Determines the PWM frequency
- **($f_{PWM} = 1/T_{Period}$).**
  - **Duty Cycle:** The percentage of the period that the signal is ON (HIGH).

The Formula for Duty Cycle:

$$\text{Duty Cycle (\%)} = \frac{T_{ON}}{T_{Period}} \times 100\%$$

Relationship to Average Voltage: Relationship to Average Voltage: The average DC voltage of a PWM signal is directly proportional to its duty cycle.

$$V_{avg} = V_{high} \times \text{Duty Cycle}$$

Example: **A 5V** signal at **20%** duty cycle has an average voltage of **5V×0.20=1V.**



RENESAS

Altium

The Engineering Trade-off: Performance vs. Power

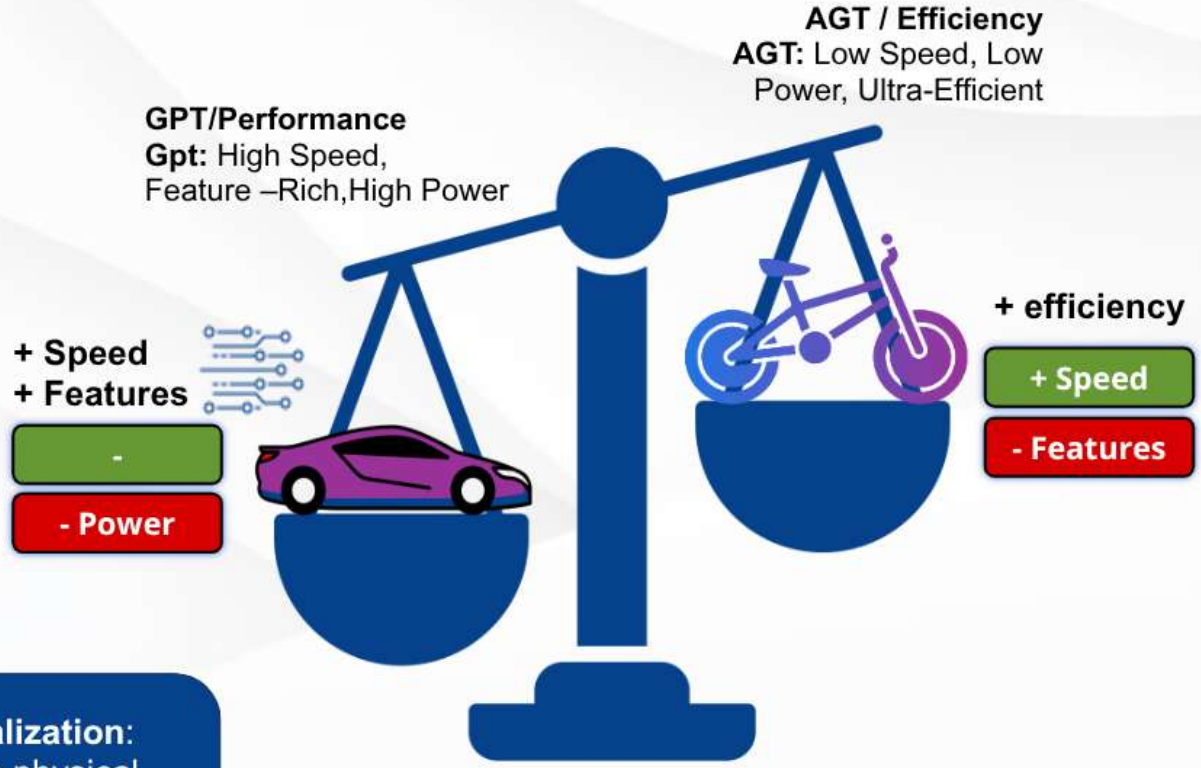
Why have both GPT and AGT?
They represent a fundamental design trade-off.

GPT (Generalist):

- **Pro**: High speed, high resolution, feature-rich (e.g., advanced PWM).
- **Con**: High power consumption; tied to the main system clock.

AGT (Specialist):

- **Pro**: Extremely low power consumption; can operate in deep sleep modes.
- **Con**: Lower speed, lower resolution, simpler feature set.



The Principle of Optimization Through Specialization: The existence of both timers on a single chip is a physical manifestation of the engineering need to balance performance with power efficiency, a central challenge in modern embedded design.

RENESAS

Altium

How it Works: AGT Calculation (Periodic Wake-up)

Step 1: Identify the Source Clock Frequency (f_{clk})

$$f_{clk} = 32,768 \text{ Hz}$$

Step 2: Choose a Prescaler. The AGT offers a wide prescaler range for low-speed clocks. To achieve a long delay with a 16-bit counter, we need a large division. Let's choose 128.23

Step 3: Calculate the AGT's Clock Frequency.

$$f_{AGT} = f_{clk}/\text{Prescaler} = 32,768Hz/128 = 256Hz$$

Step 4: Calculate the Required Number of Ticks (Period Value)

$$\text{Required Ticks} = T_{timer} \times f_{AGT} = 10 \text{ s} \times 256 \text{ Hz} = 2,560$$

- This value is loaded into the AGT's period register. It fits easily within a 16-bit counter (max 65,535).



RENESAS

Altium

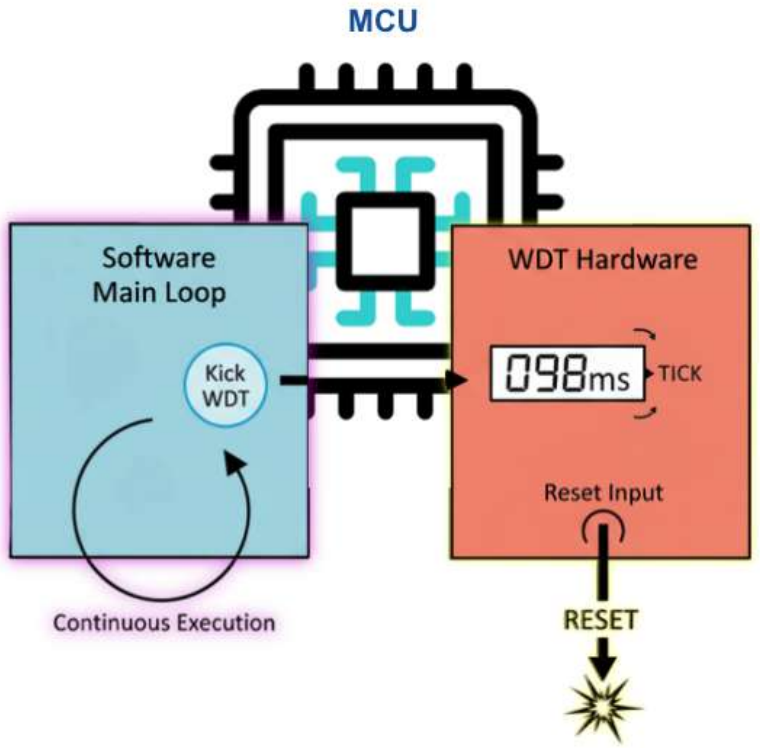
The Guardian: Watchdog Timer (WDT)

Definition: A hardware safety mechanism designed to automatically detect and recover from software malfunctions (e.g., freezes, infinite loops).

It's a "**Dead Man's Switch**": The system is assumed to be running correctly only if it is able to perform a specific action periodically.

The Core Principle: "**Kicking the Dog**"

1. The WDT is a timer that is always counting down.
2. The main application software MUST periodically reset the WDT's counter before It reaches zero. This action is called "**kicking**" or "**feeding**" the dog.
3. If the software hangs, it fails to kick the watchdog.
4. The WDT times out and triggers a corrective action, usually a full system **RESET**.



RENESAS

Altium

Fail-Safe Design: Internal vs. External WDTs

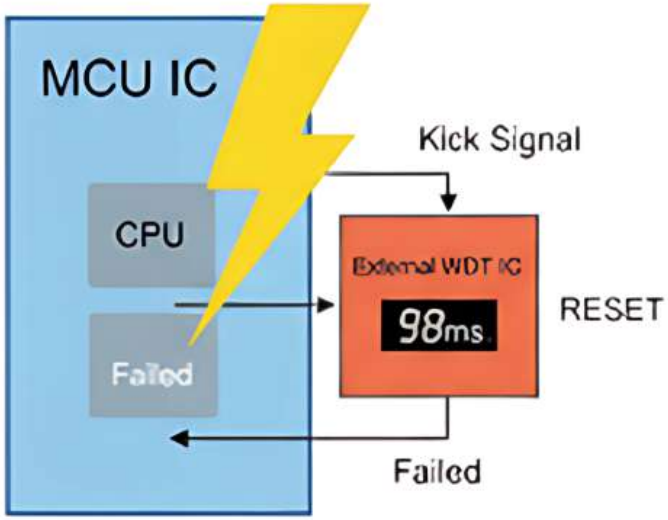
Click the numbered buttons to learn more.

Internal WDT:

Location: Built into the microcontroller silicon.

Pros: No extra cost or board space.

Cons: Can be vulnerable to the same faults that affect the CPU. A software bug or power glitch could potentially disable the WDT itself (a "**Common-Cause Failure**").



Internal WDT shares CPU power & silicon.

1 2

RENESAS

Altium

Fail-Safe Design: Internal vs. External WDTs

Click the numbered buttons to learn more.

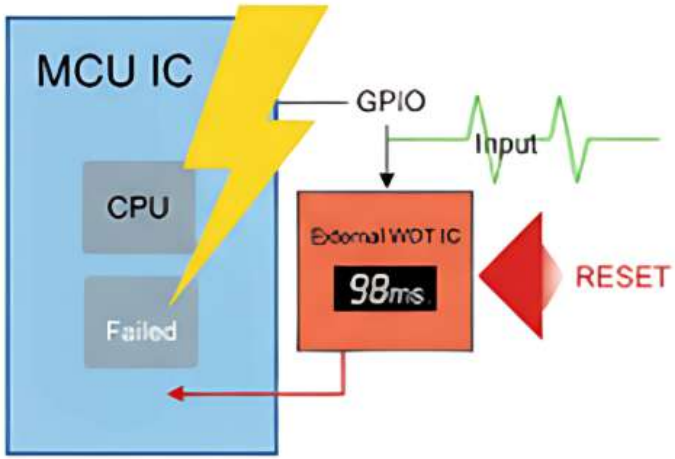
External WDT:

Location: A separate IC on the circuit board.

Pros: Provides true redundancy. It operates independently and is immune to the MCU's internal software state, making it far more reliable for safety-critical applications.

Cons: Adds cost and complexity.

This embodies the principles of Fail-Safe Design and Redundancy, which are the bedrock of functional safety standards like ISO 26262 for automotive systems.



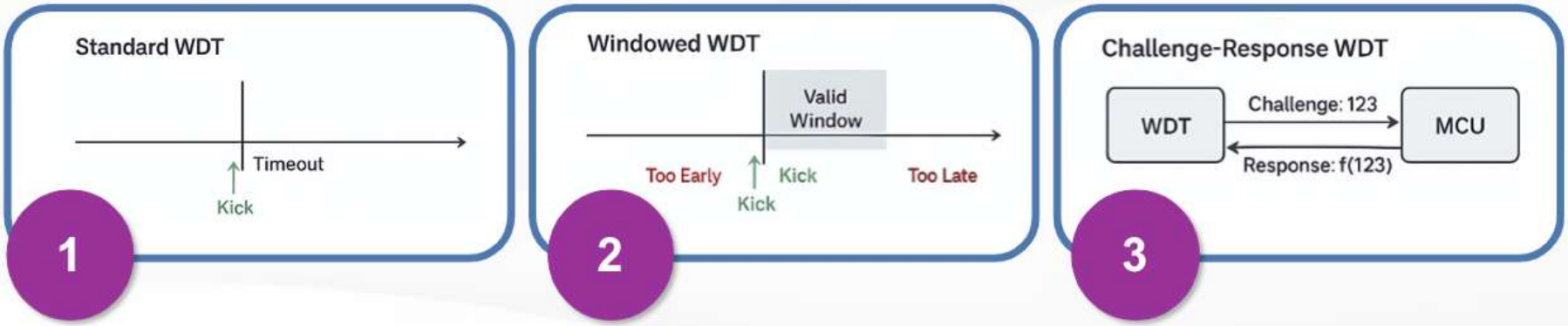
WDT operates independently, true redundancy.

1 2

RENESAS Altium

Advanced WDT Concepts

- **Standard WDT:** Detects if the kick is too late (a system freeze).
- **Windowed WDT:** A more advanced type that defines a time window. The kick is only valid if it occurs inside this window.
  - **Too Late**: A kick after the window closes is a fault (system freeze).
  - **Too Early:** A kick before the window opens is also a fault. This can detect a task that is stuck in a tight, fast loop but is not completing its full set of duties.
- **Challenge-Response WDT:** The most secure type.
  1. The WDT sends a random number (the "challenge") to the MCU.
  2. The MCU must perform a specific calculation on that number and write the correct result back (the "response").
  3. This verifies not only that the code is running, but that it is executing correctly and hasn't been corrupted.



RENESAS

Altium

Application Deep Dive: LED Control with Compare Match

Click the numbered buttons to learn more.

GPT's Role (as a Compare Match Timer):

- A GPT is configured in Periodic Mode to generate an interrupt at a fixed interval (e.g., every 250ms).
- The timer's counter increments based on a divided peripheral clock.
- When the counter value matches the period register (a compare match), the timer overflows and triggers an interrupt.
- The Interrupt Service Routine (ISR) is a small, fast piece of code that executes automatically. Its only job is to invert the state of the output pins connected to the LEDs.
- This leaves the main CPU loop completely free to handle other tasks, like responding to user button presses.

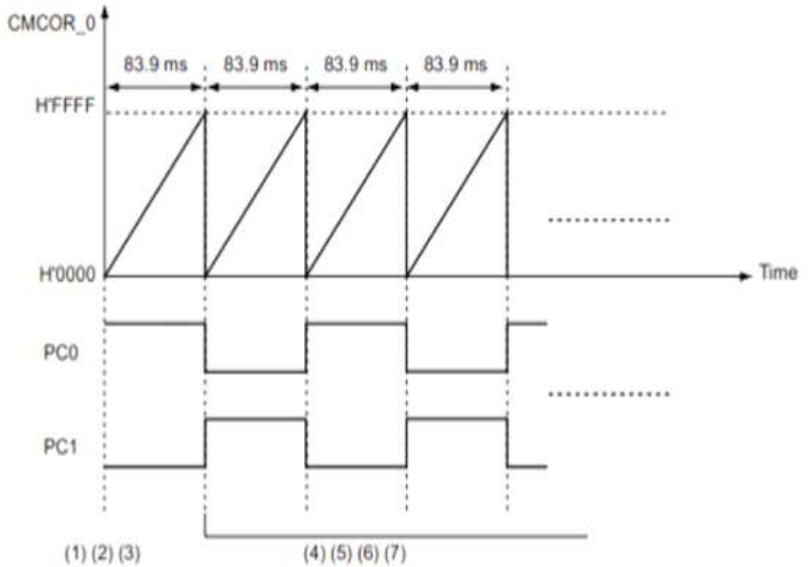


Figure 5 CMT Interrupt and Port Outputs

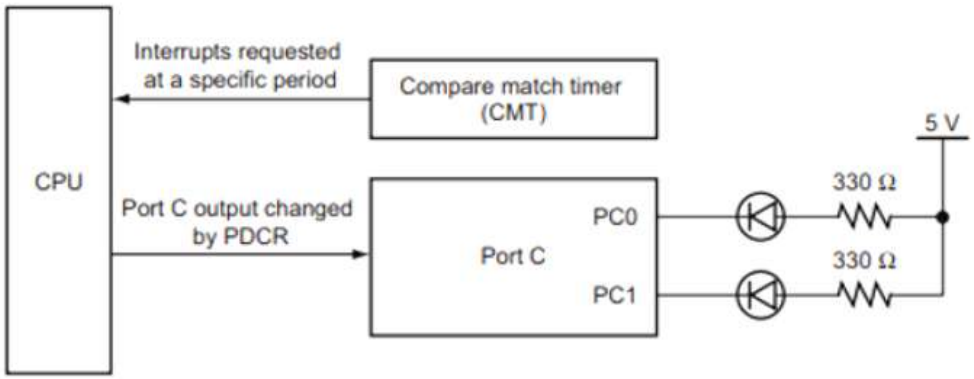
1 2

Application Deep Dive: LED Control with Compare Match

Click the numbered buttons to learn more.

Application: Blinking two LEDs alternately on a **Renesas RA6E2 Quickconnect kit**. A foundational "hello world" for timers.

The Challenge: Create a precise 500ms blink rate (250ms on, 250ms off for each LED, alternating) without using blocking delay() functions, which would freeze the entire application.



1 2

RENESAS

Altium

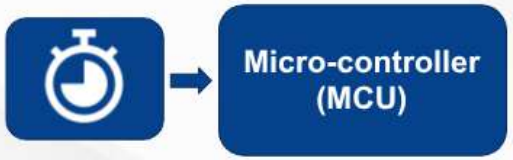
The Chronicler: Real-Time Clock (RTC)

Definition: A specialized timer that keeps track of the current time of day and date in a human-readable format (seconds, minutes, hours, day, month, year).

Most Critical Feature: Power Independence

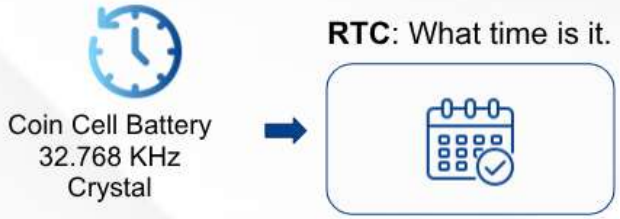
- The RTC operates from a separate power source (a coin cell battery or supercapacitor) and a dedicated low-frequency crystal.
- This allows it to maintain the correct time and date for years, even when the main system power is completely off.

General Purpose Timer (GPT)



GPT: How long dit it take?

Real Time Clock (RTC)



RTC: What time is it.

Separate Power Source

RENESAS Altium

How it Works: The 32.768 kHz Crystal

The Perfect Frequency: Most RTCs use a 32.768 kHz quartz crystal oscillator, the same kind found in digital watches.

Why this specific number?

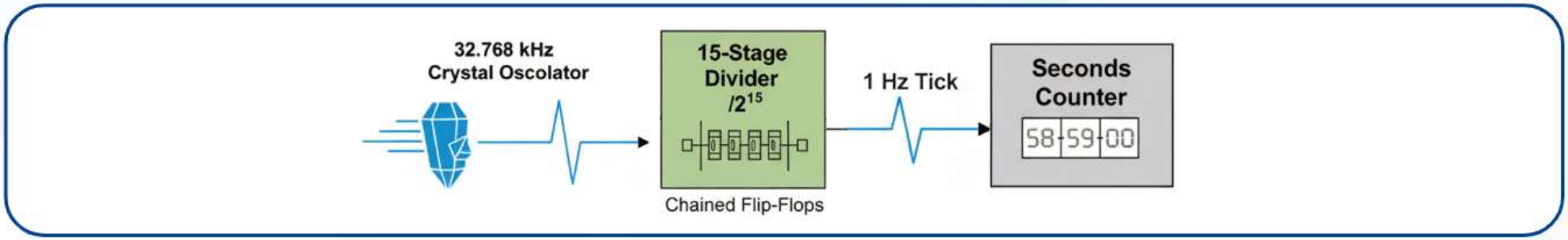
- $32{,}768 = 2\ ^{15}$
- This is a brilliant piece of engineering. A simple, low-power 15-stage binary counter (a chain of 15 flip-flops) can divide this frequency down to produce a perfect 1 Hz signal (one pulse per second).

Data Format: BCD (Binary Coded Decimal)

- RTC registers often store time in BCD format, where each decimal digit is encoded in

4 bits.

- **Example**: The number 23 is stored as $0010\ 0011$ in BCD, not $0001\ 0111$ as in normal binary.
- This simplifies the conversion to ASCII characters for display.



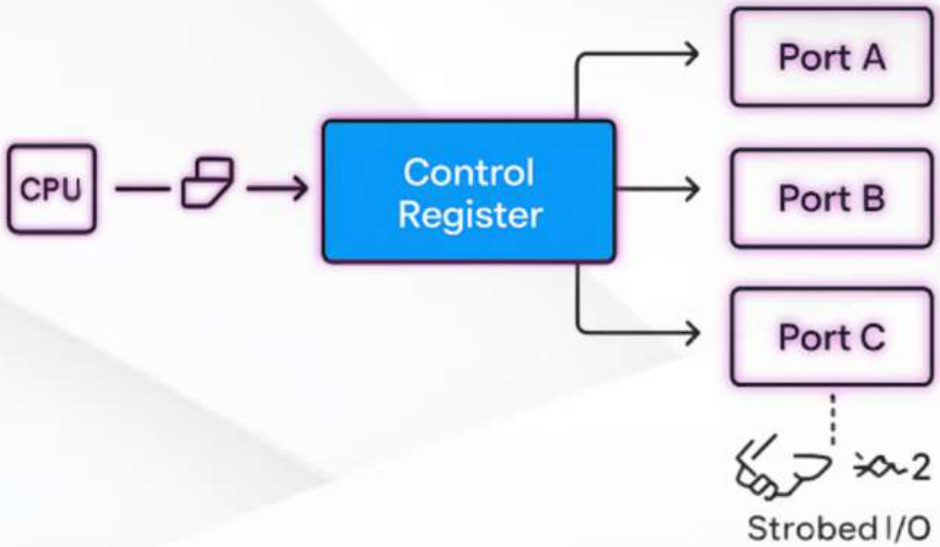
RENESAS

Altium

Deeper Dive: The Intel 8255 PPI

Key Features:

- Provided 24 programmable I/O pins, grouped into three 8-bit ports (Port A, Port B, Port C).
- **Software Configurable**: A "Control Word" was written by the CPU to a register to define the mode and direction of each port.
- **Three Modes of Operation:**
  - **Mode 0 (Basic I/O):** Simple input or output.
  - **Mode 1 (Strobed I/O):** Ports A and B use lines from Port C for handshaking signals.
  - **Mode 2 (Strobed Bidirectional Bus):** Port A becomes a bidirectional bus, using Port C for handshaking.



Legacy: "The functionality of the 8255 is now mostly embedded in larger VLSI processing chips as a sub-function." **A CMOS version is still made by Renesas!**

Course Content	
CONTINUOUS AND DISCRETE TIME SIGNALS AND SYSTEMS Continuous Time (CT) &Discrete Time (DT) signals- Classification - standard signals – basic operations on signals - Continuous time and discrete time systems - properties - Linear Time Invariant (LTI) systems - Stability - Causality - Convolution Integral, Convolution sum.	**16 Hours**